11-785: Introduction to Deep Learning

HW3P1 Bootcamp

RNNs, GRUs, CTC, and CTC Decoding

Manigandan Ramadasan, Ron Sarma, Felix Hirwa Nshuti, Sid

February 28, 2026

Logistics

- Early Submission : **March 20th, 11:59pm**
- On-Time Submission: **March 27th, 11:59pm**

No late days can be used for Homework part 1s, please plan accordingly!

Homework Objectives

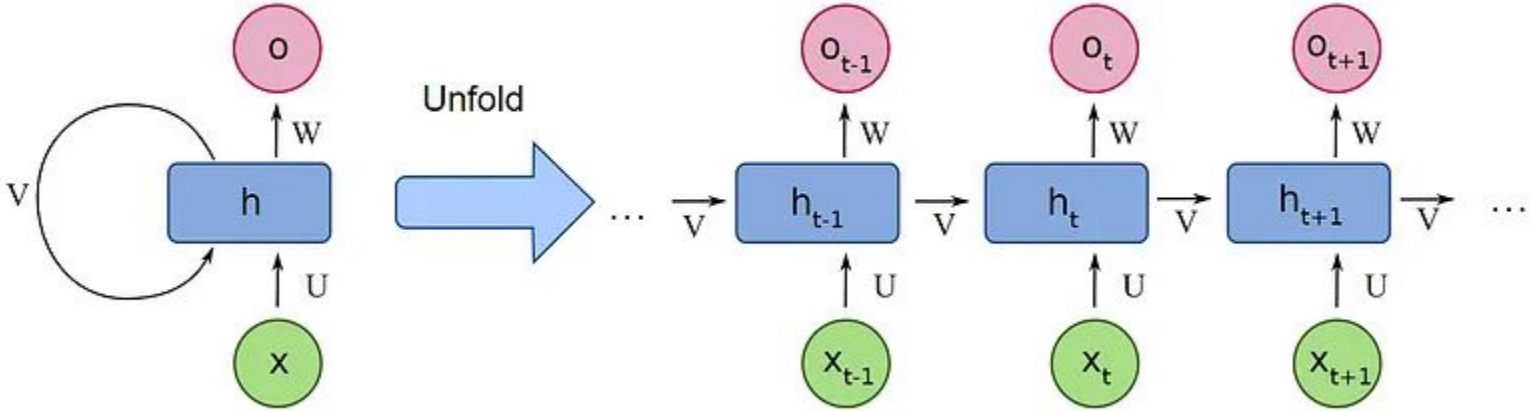
- Implement **RNN** cells
- Build a simple **RNN Classifier** using the RNN cells
- Implement **GRU** cells
- Build a **character predictor** using GRU cells
- Implement **CTC loss** forward and backward pass
- Implement **Greedy Search** and **Beam Search** for decoding

```
HW3P1
├── mytorch
│   ├── nn
│   │   ├── linear.py
│   │   ├── activation.py
│   │   └── loss.py
│   ├── gru_cell.py
│   ├── rnn_cell.py
│   └── utils.py
├── models
│   ├── char_predictor.py
│   └── rnn_classifier.py
├── MCQ
│   └── mcq.py
├── CTC
│   ├── CTC.py
│   └── CTCDecoding.py
├── autograder
│   └── runner.py
└── create_tarball.sh
```

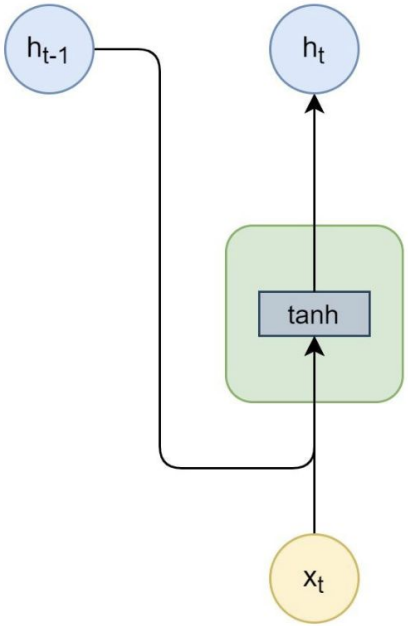
Recurrent Neural Networks (RNNs)

Structure of RNNs

A simple RNN that does seq-to-seq task with one RNN cell



RNN Cell Forward and Backward

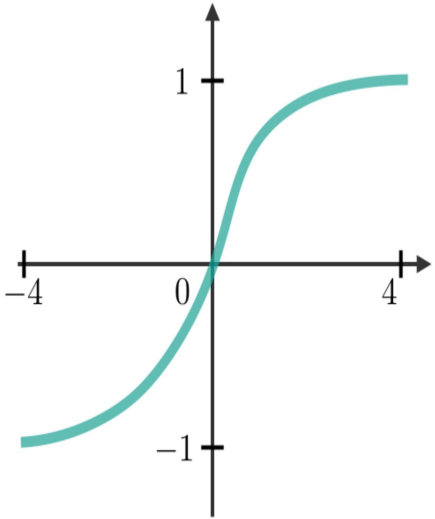


$$h_t = tanh(W_{ih} x_t + b_{ih} + W_{hh} h_{t-1} + b_{hh})$$

Very similar to `linear.py` in HW1P1.

We use $tanh$ function to linear transformations of x_t and h_{t-1}

Why tanh?

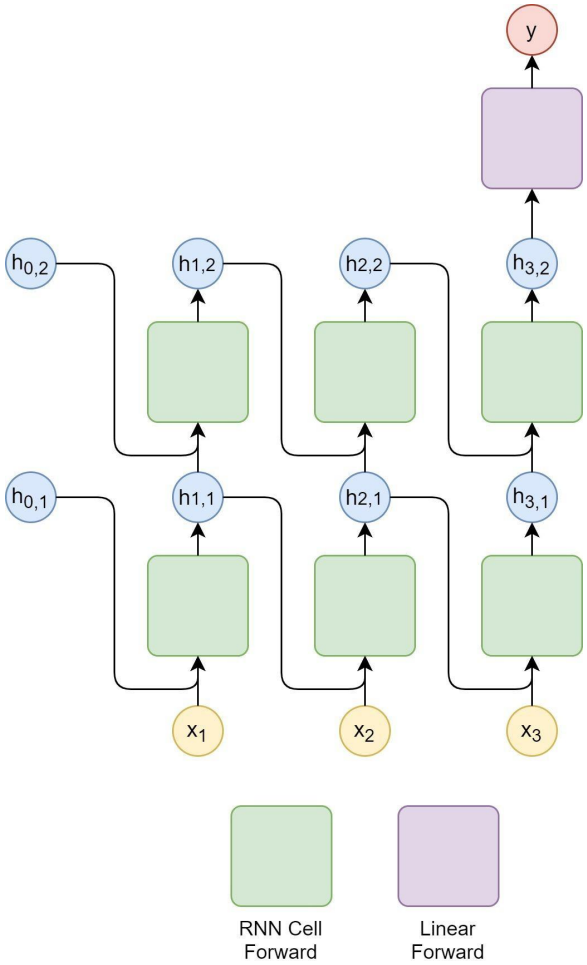


$$h_t = \boxed{tanh}(W_{ih}x_t + b_{ih} + W_{hh}h_{t-1} + b_{hh})$$

1. Non-linearity
2. Tanh is bounded; can mitigate exploding gradient problem

RNN Phoneme Classifier

- **Many-to-one** task
- Input sequence is passed through a few layers of **RNN cells**
- The final hidden state at the final timestamp is passed through a **linear layer** to give us the phoneme class



Gated Recurrent Units (GRUs)

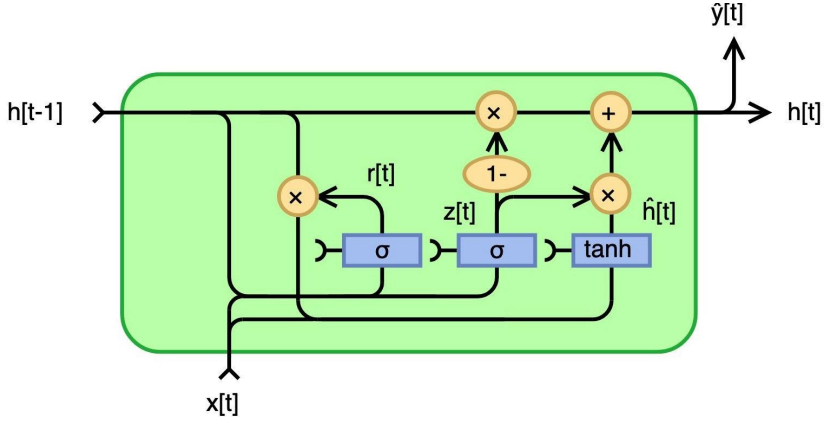
GRU Cell Forward and Backward

Reset Gate: $\mathbf{r}_t = \sigma(\mathbf{W}_{rx} \cdot \mathbf{x}_t + \mathbf{b}_{rx} + \mathbf{W}_{rh} \cdot \mathbf{h}_{t-1} + \mathbf{b}_{rh})$

Update Gate: $\mathbf{z}_t = \sigma(\mathbf{W}_{zx} \cdot \mathbf{x}_t + \mathbf{b}_{zx} + \mathbf{W}_{zh} \cdot \mathbf{h}_{t-1} + \mathbf{b}_{zh})$

Memory Content: $\mathbf{n}_t = tanh(\mathbf{W}_{nx} \cdot \mathbf{x}_t + \mathbf{b}_{nx} + \mathbf{r}_t \odot (\mathbf{W}_{nh} \cdot \mathbf{h}_{t-1} + \mathbf{b}_{nh}))$

Hidden State: $\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{n}_t + \mathbf{z}_t \odot \mathbf{h}_{t-1}$



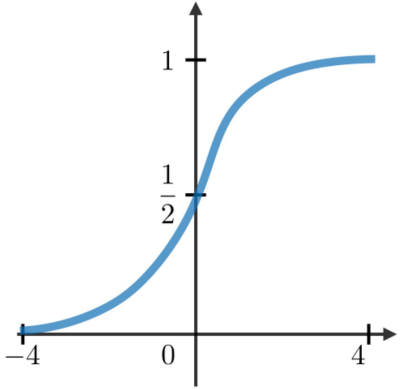
GRU Cell Forward and Backward contd.

Reset Gate: $\mathbf{r}_t = \boxed{\sigma}(\mathbf{W}_{rx} \cdot \mathbf{x}_t + \mathbf{b}_{rx} + \mathbf{W}_{rh} \cdot \mathbf{h}_{t-1} + \mathbf{b}_{rh})$

Update Gate: $\mathbf{z}_t = \boxed{\sigma}(\mathbf{W}_{zx} \cdot \mathbf{x}_t + \mathbf{b}_{zx} + \mathbf{W}_{zh} \cdot \mathbf{h}_{t-1} + \mathbf{b}_{zh})$

Memory Content: $\mathbf{n}_t = tanh(\mathbf{W}_{nx} \cdot \mathbf{x}_t + \mathbf{b}_{nx} + \mathbf{r}_t \odot (\mathbf{W}_{nh} \cdot \mathbf{h}_{t-1} + \mathbf{b}_{nh}))$

Hidden State: $\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{n}_t + \mathbf{z}_t \odot \mathbf{h}_{t-1}$



Why sigmoid?

Sigmoid is limited to the values 0 to 1

→ This describes how much info to pass

→ Close to 0: we want to “forget”

→ Close to 1: we want to “remember”

GRU Cell Forward and Backward contd.

Reset Gate: $\mathbf{r}_t = \sigma(\mathbf{W}_{rx} \cdot \mathbf{x}_t + \mathbf{b}_{rx} + \mathbf{W}_{rh} \cdot \mathbf{h}_{t-1} + \mathbf{b}_{rh})$

Update Gate: $\mathbf{z}_t = \sigma(\mathbf{W}_{zx} \cdot \mathbf{x}_t + \mathbf{b}_{zx} + \mathbf{W}_{zh} \cdot \mathbf{h}_{t-1} + \mathbf{b}_{zh})$

Memory Content: $\mathbf{n}_t = tanh(\mathbf{W}_{nx} \cdot \mathbf{x}_t + \mathbf{b}_{nx} + \mathbf{r}_t \odot (\mathbf{W}_{nh} \cdot \mathbf{h}_{t-1} + \mathbf{b}_{nh}))$

Hidden State: $\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{n}_t + \mathbf{z}_t \odot \mathbf{h}_{t-1}$

The $\odot$ operator here is a Hadamard product (elementwise)
The $\cdot$ operator here is a matrix multiplication (you can use np.matmul)

GRU Cell Forward and Backward contd.

Reset Gate: $\mathbf{r}_t = \sigma(\mathbf{W}_{rx} \cdot \mathbf{x}_t + \mathbf{b}_{rx} + \mathbf{W}_{rh} \cdot \mathbf{h}_{t-1} + \mathbf{b}_{rh})$

Update Gate: $\mathbf{z}_t = \sigma(\mathbf{W}_{zx} \cdot \mathbf{x}_t + \mathbf{b}_{zx} + \mathbf{W}_{zh} \cdot \mathbf{h}_{t-1} + \mathbf{b}_{zh})$

Memory Content: $\mathbf{n}_t = tanh(\mathbf{W}_{nx} \cdot \mathbf{x}_t + \mathbf{b}_{nx} + \boxed{\mathbf{r}_t \odot (\mathbf{W}_{nh} \cdot \mathbf{h}_{t-1} + \mathbf{b}_{nh})})$

Hidden State: $\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{n}_t + \mathbf{z}_t \odot \mathbf{h}_{t-1}$

We do an element-wise product with a linear transformation of the previous hidden-state. We are combining the transformation of the inputs from candidate space with the information we are retaining from previous hidden state.

GRU Cell Forward and Backward - Tips

- The longest question in HW3P1
- All intermediate **dWs** and **dbs** should be correct to make sure that your **dx** and **dh** are correct
- Useful resource: [How to compute a derivative]
- Break down complicated equations into unary/binary operations
- e.g. $f(x) = \tanh(r \odot (Wx+b))$, we want to decompose it into:
  - $Z1 = Wx + b$
  - $Z2 = r \odot (Wx+b)$
  - $f(x) = \tanh(Z2)$
  - Finally we apply chain rule to get $f'(x)$

GRU Cell Forward and Backward - Tips

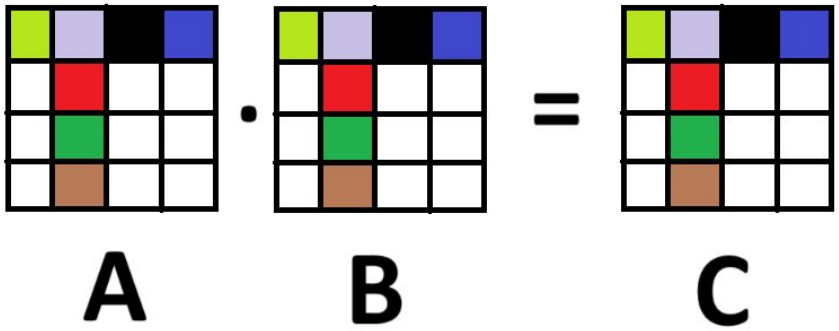
- **Expected shapes**: both r_t, z_t, n_t, and h_t have the shape (batch_size, hidden_size)
- **Bias Broadcasting:** Bias parameters are of shape (hidden_size,). During addition, they implicitly broadcast across the batch_size dimension to match (batch_size, hidden_size)
- As said above order of operations matters.

Chain rule through element-wise multiplication

Assume that the shape of derivative wrt a matrix is the same as that of the matrix.

Let $C = A \odot B$ (element-wise)

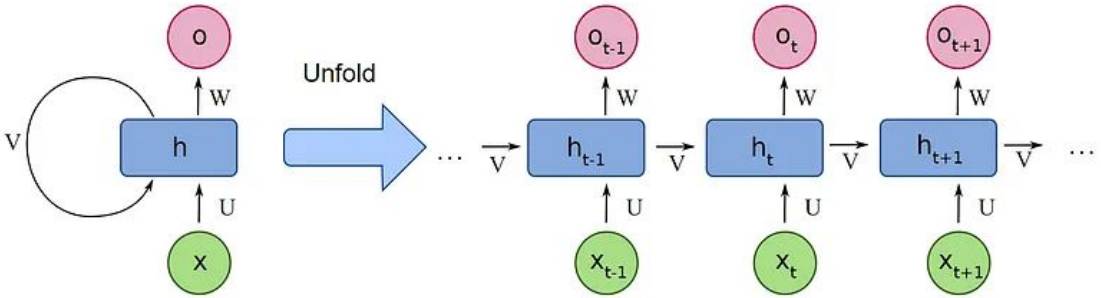
- This means A, B, and C have the same shape
- Only elements of the same position are related to each other
  $\rightarrow$ derivatives flow only position-wise
- Therefore, $dLdA = dLdC \odot B$ and $dLdB = dLdC \odot A$



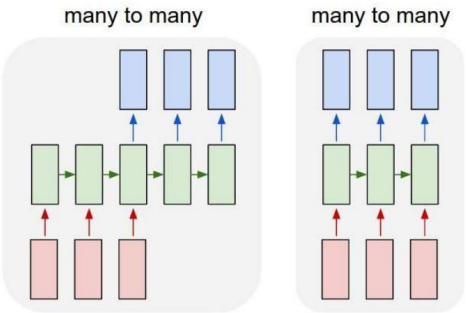
GRU Inference - Character Predictor

Many-to-many task, the model is supposed to have an output for each timestamp.

Different from RNN Phoneme classifier, here we need to pass the hidden state at **each** timestamp to a linear layer to predict the character at each t, instead of just the previous timestamp's hidden state.



Variants



Images from Karpathy

- 1: *Delayed* sequence to sequence, e.g. machine translation
- 2: Sequence to sequence, e.g. stock problem, label prediction
- Etc…

Connectionist Temporal Classification (CTC)

Connectionist Temporal Classification (CTC)

1) Why CTC?
2) How does it work?
3) Implementation in HW3 P1

Why CTC?

- CTC is used to calculate loss when our input sequences and output labels have **different lengths and no fixed alignment.**
- This method allows models to learn without needing pre-aligned data, which is crucial for applications like **speech recognition.**

Input (from RNN over time) → "hhheelllooo" (length = 10)

Target sequence → "hello"

Implementation [CTC/CTC.py]

- CTC
  - extend_target_with_blank()
  - get_forward_probs()
  - get_backward_probs()
  - get_posterior_probs()
- CTCLoss

1. Extend target with blank

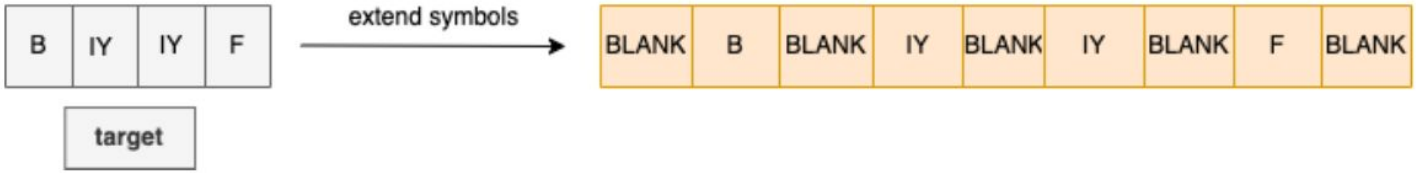


Figure 13: Extend symbols

1. Extend target with blank

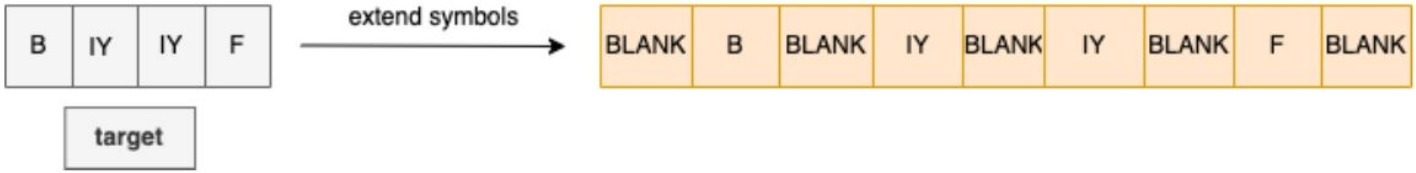


Figure 13: Extend symbols

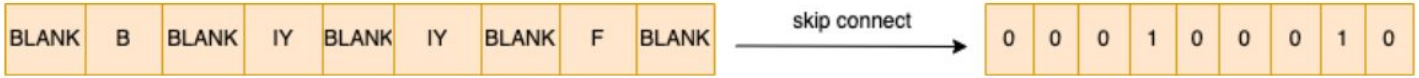
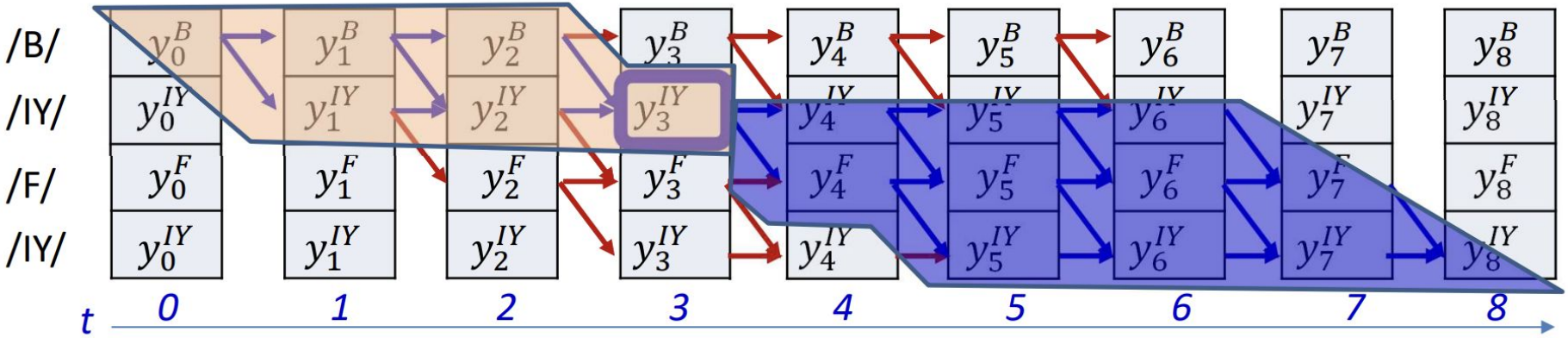


Figure 14: Skip connections

2. Forward Probabilities

$$\alpha(t,r) = P(S_0 ... S_r, s_t = S_r | X) = \sum_{q:S_q \in pred(S_r)} \alpha(t-1,q) y_t^{S_r}$$



2. Forward Probabilities

$$\alpha(t,r) = P(S_0 ... S_r, s_t = S_r | X) = \sum_{q:S_q \in pred(S_r)} \alpha(t-1,q) y_t^{S_r}$$

Dynamic programming table

alpha

	t = 1	t = 2	t = 3	t = 4	t = 5	t = 6
BLANK	0	0	0	0	0	0
B	.1	.05	.045	.0135	.00945	.000945
BLANK	0	.04	.072	.0234	.02214	0
IY	0	.06	0	.0468	.06696	.01971
BLANK	0	0	.048	.0096	.03384	0
IY	0	0	0	.0192	.02304	.011376
BLANK	0	0	0	0	.01152	0
F	0	0	0	0	.01728	.015552
BLANK	0	0	0	0	0	0

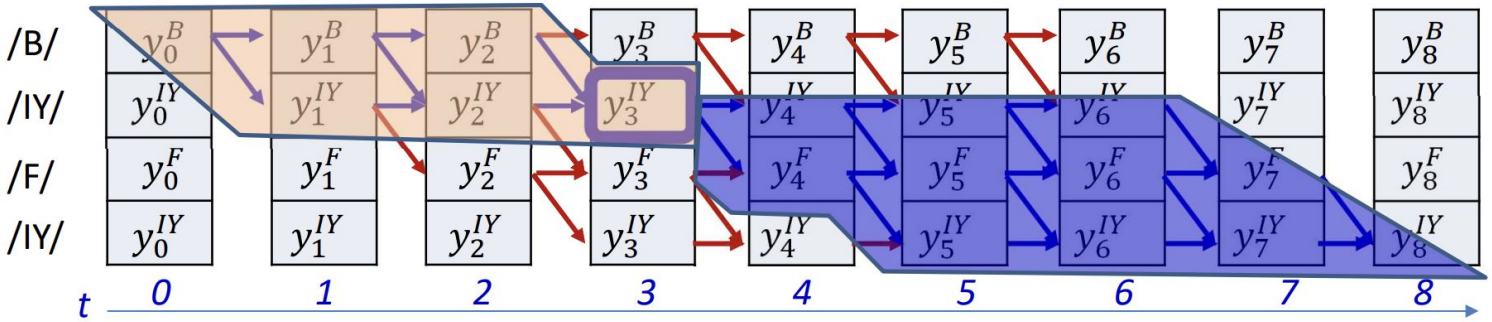
input: x1 → x2 → x3 → x4 → x5 → x6

Figure 15: Forward Algorithm

3. Backward Algorithm

- Computing probabilities from right to left
- **β(t, r)** represents probability of generating the rest of the sequence starting from time t

$$\beta(t,r) = P(s_{t+1} \in succ(S_r), succ(S_r), ..., S_{K-1}|X) = \sum_{q:S_q \in succ(S_r)} \beta(t+1,q) y_{t+1}^{S_q}$$



4. CTC Posterior Probability

- Represents probability of being in state **r** at time **t** given input sequence.

$$\gamma(t,r) = P(s_t = S_r | S, X) = \frac{\alpha(t,r)\beta(t,r)}{\sum_{r'} \alpha(t,r)\beta(t,r)}$$

5. CTC Loss Computation

- Once we know the posterior probability, we can plug it in to calculate the loss, which will ultimately be used to train your network.

$$\gamma(t,r) = P(s_t = S_r | S, X) = \frac{\alpha(t,r)\beta(t,r)}{\sum_{r'} \alpha(t,r)\beta(t,r)}$$

$$DIV = -\sum_{t} \sum_{s \in S_0 \ldots S_{K-1}} P(s_t = s | \mathbf{S}, \mathbf{X}) \log Y(t, s_t = s)$$

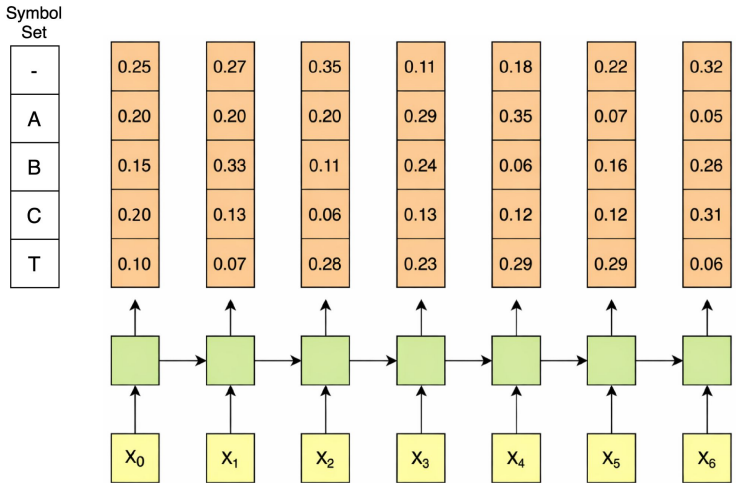
$$DIV = -\sum_{t} \sum_{r} \gamma(t,r) \log y_t^{S(r)}$$

$$\frac{dDIV}{dy_t^l} = -\frac{1}{y_t^l} \sum_{r : S(r) = l} \gamma(t,r)$$

CTC Decoding
(Greedy and Beam Search)

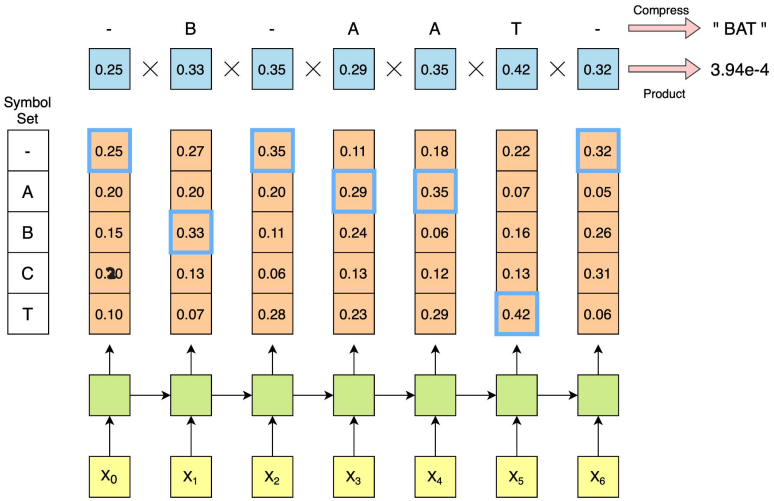
CTC Decoding

- Why Decoding?
  - CTC gives us a **probability distribution** over possible alignments.
  - We do not have the final output sequence :(
  - Need a strategy to transform the model's output into meaningful text.



CTC Decoding

- What is Decoding?
  - Process of converting these probabilities into the most likely output sequence.



- You will implement two decoding methods in ***CTC/CTCDecoding.py***
  - Greedy Search and Beam Search

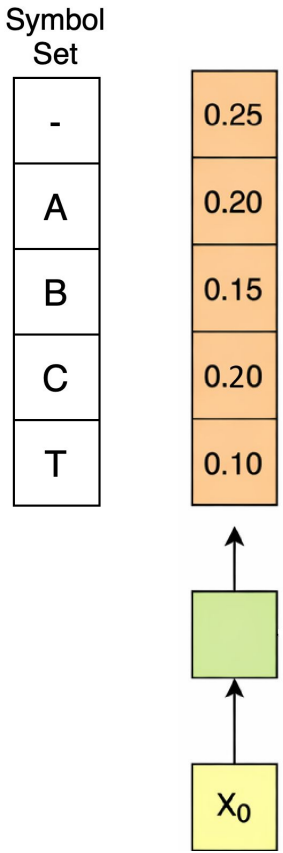
Before we go further...

Lets cover few basic terms!

(These will be covered in detail in upcoming lectures)

Preliminaries - Symbol Set

- Collection of all symbols the model can output at each time step.
- Widely known as vocabulary.
- Notice ‘ - ’ in the symbol set
  - What’s that?



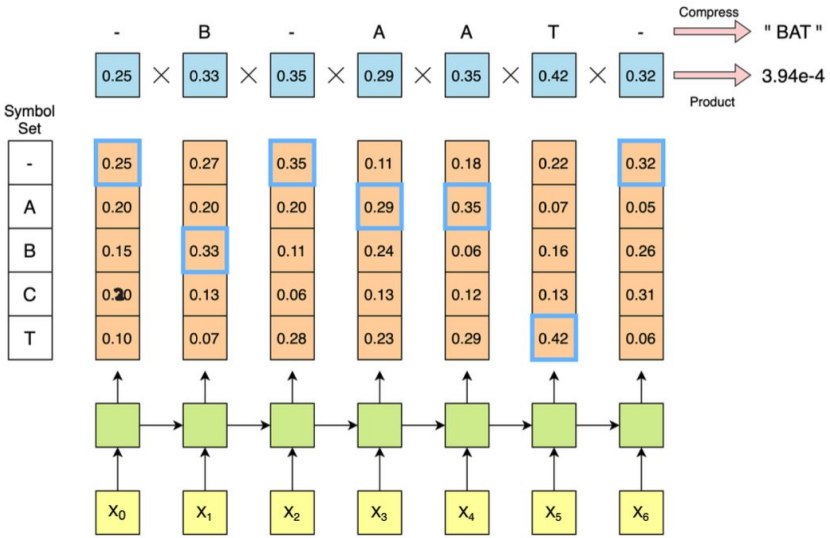
Preliminaries - Blank Symbol $\boxed{-}$

- Special symbol
- Blank symbol means no label is emitted at this time step.
- Used to avoid forcing a character at every time step
  - Example: Silence
- Compressing/Collapsing:
  - Expected Output - BAT
  - Model Output - BAAAAAT
  - BAAAAAT → BAT
- What if the expected output is HELLO and the model predicts HELLLLLO
  - Compress it to HELO?
- Blank symbol helps here- Used to separate repeated characters when needed
  - HELLL-LLO → HEL-LO → HELLO

Greedy Search

- Just be *greedy*!
- Selects the most probable symbol at each time step.
- Collapse repeated symbols and remove blanks.

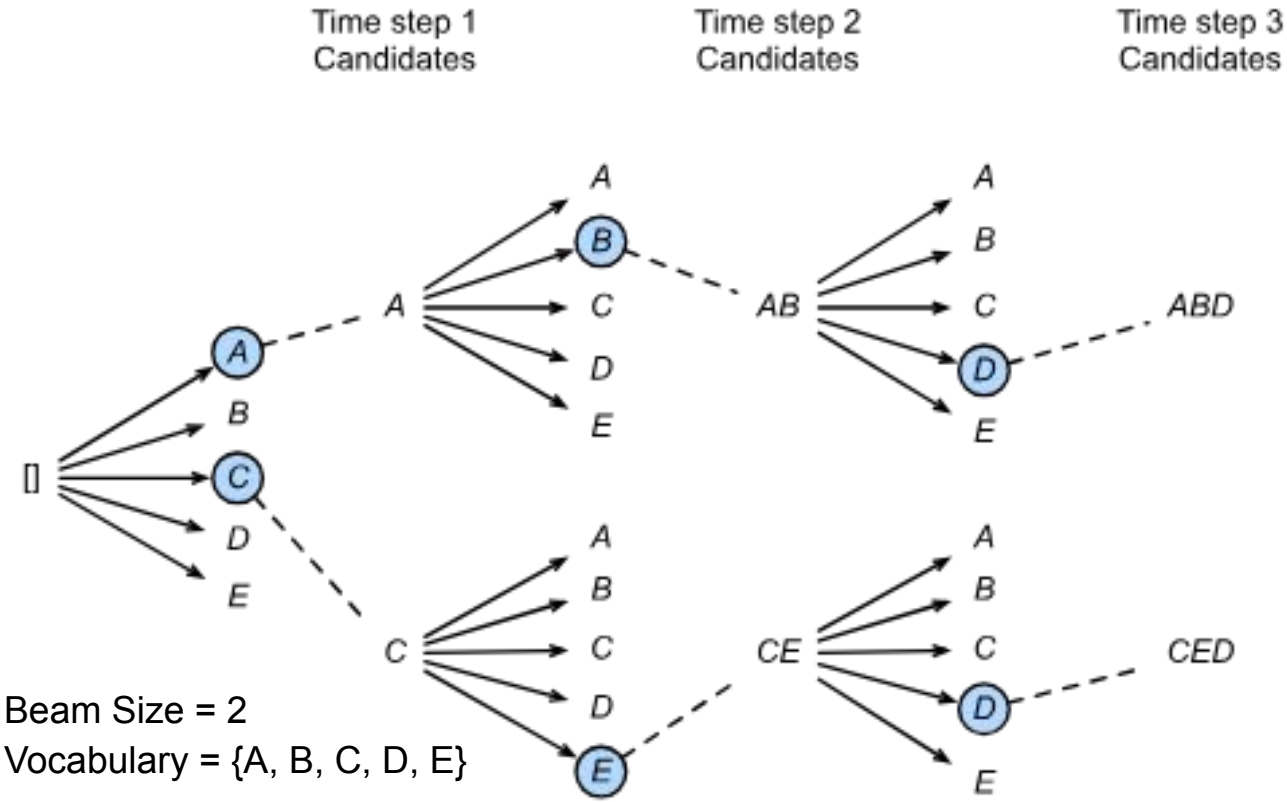
$$A^* = \arg\max_{A} \prod_{t=1}^{T} p_t(a_t|X)$$



Beam Search

- Beam search keeps the **top *k* partial sequences** at each time step.
- Instead of choosing only one best symbol, it explores multiple likely paths.
- At every time step:
  - extend each current hypothesis with possible symbols
  - compute updated scores
  - keep only top *k* (beam width) candidates
- Do not forget about collapsing and blank symbols!

(Vanilla) Beam Search



Beam Size = 2
Vocabulary = {A, B, C, D, E}

Just for the purpose of understanding beam search.

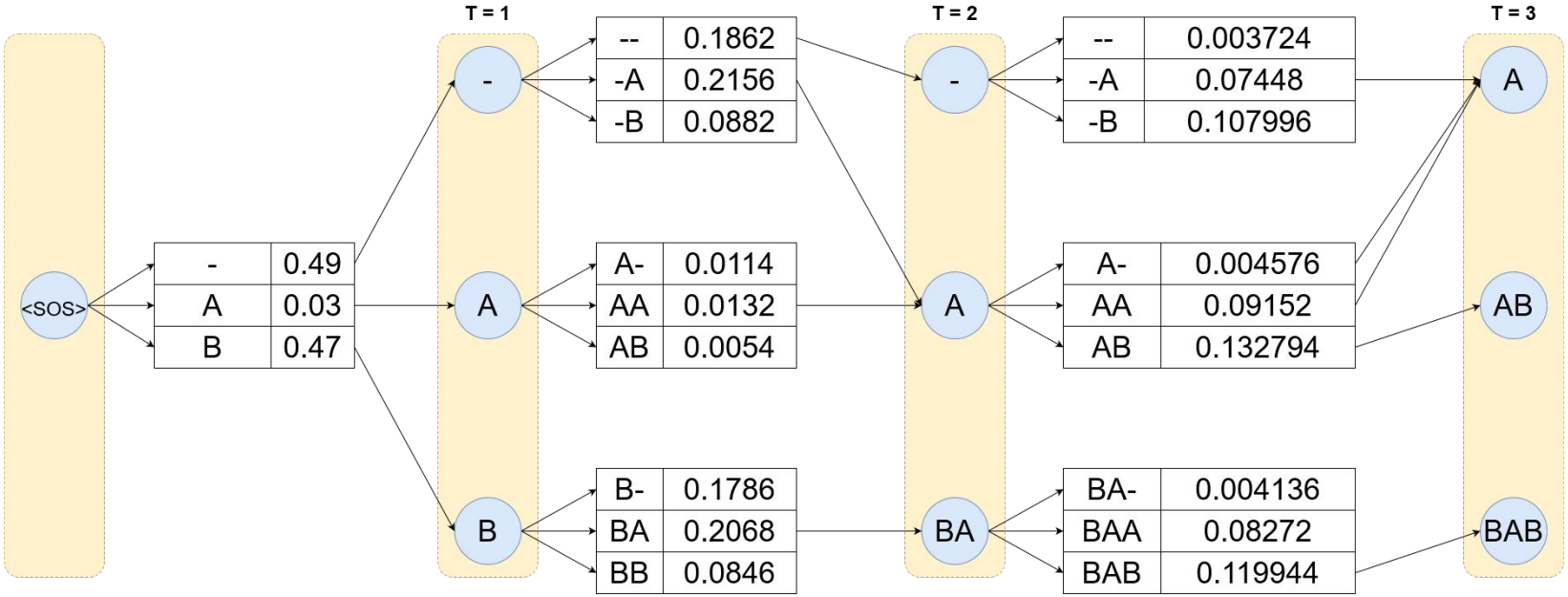
You won't implement vanilla beam search!

You will implement CTC beam search :P

(CTC) Beam Search

Beam Size = 3

Vocabulary	P(symbol) @ T=1	P(symbol) @ T=2	P(symbol) @ T=3
-	0.49	0.38	0.02
A	0.03	0.44	0.40
B	0.47	0.18	0.58



Beam Search

How to Approach this?

- Initialize the beam search with one path consisting of a 'blank' symbol. At each timestep, iterate over your current best paths.
- Extend each path by every possible new symbol, and calculate the score of the new paths.
- Remember, when extending a path with a new symbol, you'll encounter three scenarios:

  1. `The new symbol is the same as the last symbol on the path.`
  2. `The last symbol of the path is blank.`
  3. `The last symbol of the path is different from the new symbol and is not blank.`

- After extending the paths, retain only the best paths, as determined by the `beam_width`.
- At the end of all timesteps, remove the 'blank' symbol if it's at the end of a path. Translate the numeric symbol sequences to string sequences, sum up the scores for paths that end up with the same final sequence, and find the best overall path.
- Return the highest scoring path and the scores of all final paths.

You can implement additional methods within this class, as long as you return the expected variables from the decode method (which is called during training). We recommend following the below pseudocode.

Beam Search Pseudocode

Algorithm 2: Beam Search Decoding Algorithm

Input: $SymbolSet, y_probs, BeamWidth$
Output: $BestPath, MergedPathScores$

```
Initialization:  Create an empty dictionary ActivePaths with an initial blank path mapped to a
  score of 1.0   Create an empty dictionary TempPaths for temporary storage during each timestep
foreach timestep in y_probs do
    Extract the symbol probabilities at the current timestep;
    Sort ActivePaths by score in descending order and keep only the top BeamWidth paths;
    foreach (path, score) in ActivePaths do
        foreach symbol in SymbolSet including blank do
            Compute the new path based on the last symbol of path;
            Calculate the updated score: new_score = score × probability(symbol);
            if new_path exists in TempPaths then
                Add new_score to the existing score of new_path in TempPaths;
            end
            else
                Store new_score in TempPaths with new_path;
            end
        end
    end
    Replace ActivePaths with TempPaths;
    Clear TempPaths for the next timestep;
end
```

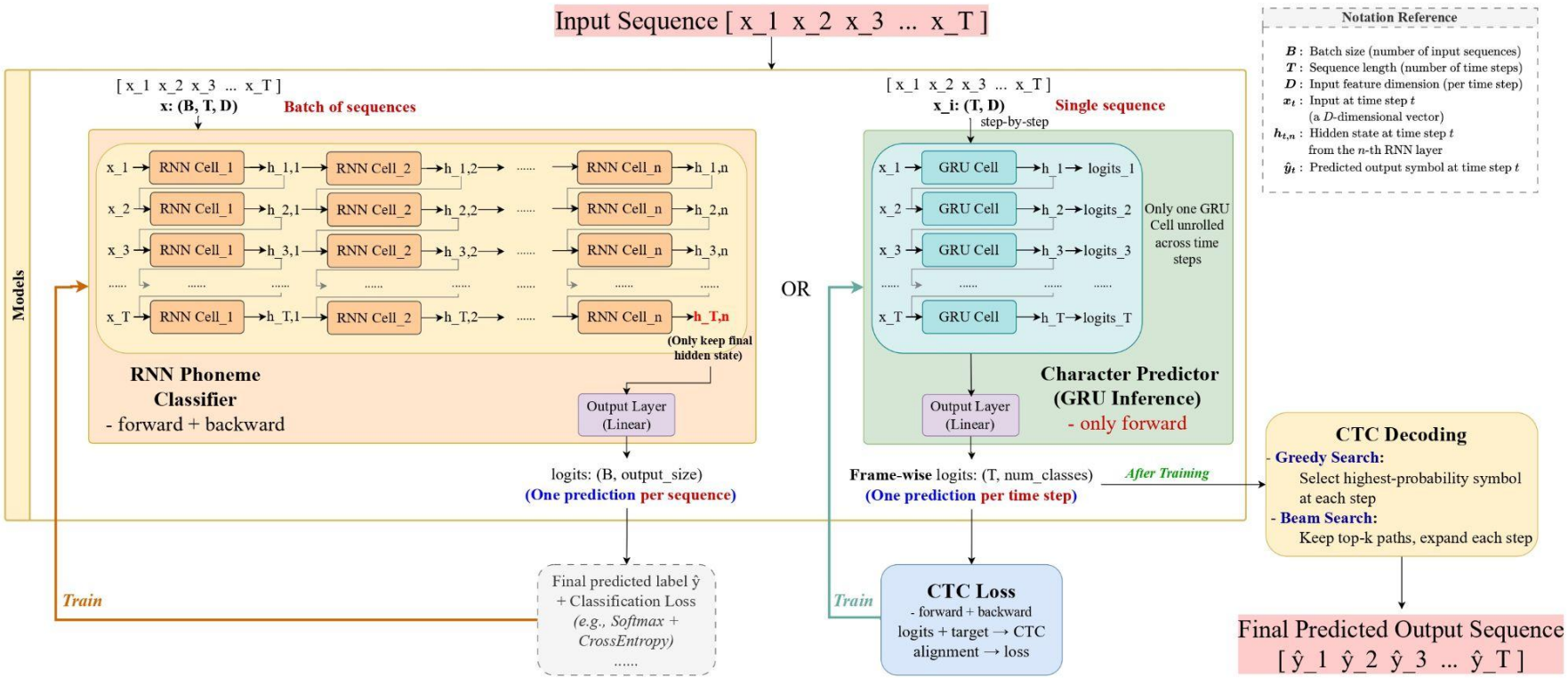
- Remember, when extending a path with a new symbol, you'll encounter three scenarios:

  1. The new symbol is the same as the last symbol on the path.
  2. The last symbol of the path is blank.
  3. The last symbol of the path is different from the new symbol and is not blank.

Beam Search Pseudocode (Contd.)

```
Final Merging of Paths:   Initialize BestPath as an empty string and BestScore as 0;
foreach (path, score) in ActivePaths do
    Remove empty spaces at the beginning and end of the path;
    if path in MergedPaths then
        Add score to path in MergedPathScores;
    end
    else
        Store score in new path in MergedPathScores;
    end
    if score is greater than BestScore then
        BestPath ← path;
        BestScore ← score;
    end
end
Convert Best Path to Symbols:   Convert BestPath to their corresponding symbols in SymbolSet;
  Convert all path in MergedPathScores to their corresponding symbols in SymbolSet
return BestPath, MergedPathScores;
```

Putting everything together



Questions?

Happy Spring Break :-)